

Build Your Own "Modly-style" App: A Beginner's Game Plan

Goal: Build a web app where a user uploads a picture and gets back a 3D model they can rotate and download. Later, wrap it as a desktop app with Electron.

Who this is for: A first-year CS student. Every new idea is explained the first time it appears. If a word is unfamiliar, check the glossary at the bottom.

Credit: This project is inspired by Modly by Lightning Pixel (MIT license). If you reuse any of its code, keep a visible line like *"Based on Modly by Lightning Pixel"* in your README and report.

Part 0: The big picture (read this first)

The restaurant analogy

Think of your app as a restaurant.

Restaurant	Your app	Technology
The dining room and menu	What the user sees and clicks	React (the frontend)
The waiter	Carries requests and answers back and forth	HTTP requests (the API)
The kitchen	Does the real work	Python + FastAPI (the backend)
The chef's special recipe	The AI model that makes 3D shapes	Generator (a Python function)
Order tickets	Tracks each job: waiting, cooking, done	Run manager

The customer never enters the kitchen. They place an order (upload an image), get a ticket number (a run ID), and keep asking "is it ready yet?" (polling) until the dish (the 3D file) arrives.

The flow of one request



Why not just do everything in one request?

Real AI models can take 30 seconds to several minutes. If the server made you wait on a single request, the page would freeze and the connection could time out. So we **start a job, get a ticket, and check back**. This pattern is used everywhere (video uploads, report generation, payments).

The phases at a glance

Phase	What you build	You'll know it works when...
0	Setup	Tools are installed, repo exists
1	Backend with a fake AI	You can start a job and download a file using the auto-generated docs page
2	Frontend	Clicking a button in the browser starts a job and shows progress
3	3D viewer	You see and rotate the model in the page (this is your minimum viable project)
4	Extension system	You drop a folder in and a new option appears
5	Real AI model	A real photo becomes a real mesh (needs a GPU)
6	Workflow canvas	You connect boxes with lines to build a pipeline
7	Polish	Tests, logs, README, report
8	Electron	Double-click an app instead of running two servers

Golden rule: finish each phase completely before starting the next. A small thing that works beats a big thing that doesn't.

Target folder structure

```
mymodly/
├─ api/                                # Python backend
│  └─ app/
│     ├── __init__.py                # empty file, marks a Python package
│     ├── main.py                    # the routes (URLs the frontend calls)
│     ├── runs.py                    # keeps track of jobs
│     ├── extensions.py              # loads plugins (Phase 4)
│     └─ generators/
│        ├── __init__.py            # empty
│        └─ mock.py                  # fake AI (Phase 1)
│  └─ extensions/                    # installed plugins go here
│  └─ uploads/                       # images users upload
│  └─ outputs/                       # 3D files we create
│  └─ requirements.txt               # list of Python packages
├─ web/                              # React frontend
│  └─ src/
│     ├── api/client.ts              # functions that talk to the backend
│     ├── hooks/useRun.ts            # "keep checking the job" logic
│     ├── pages/                     # Generate page, Extensions page, ...
│     └─ components/                 # Viewer, UploadBox, ...
└─ electron/                         # added in Phase 8
```

Why separate `api/` and `web/`? They are two independent programs that talk over HTTP. You can change one without breaking the other as long as the "contract" (the list of URLs below) stays the same.

The API contract (the menu)

An **API** is a list of URLs the frontend is allowed to call. Decide this first, because both sides depend on it.

Method	URL	What it does
GET	<code>/health</code>	"Are you alive?" Returns <code>{ok: true}</code>
POST	<code>/runs</code>	Upload an image and start a job. Returns <code>{id, status}</code>
GET	<code>/runs/{id}</code>	Ask about a job. Returns <code>{status, progress, error}</code>
GET	<code>/runs/{id}/result</code>	Download the finished 3D file
DELETE	<code>/runs/{id}</code>	Cancel a job
GET	<code>/extensions</code>	List installed plugins (Phase 4)

HTTP methods in one line each: GET = "give me something", POST = "here is something, do something with it", DELETE = "remove or cancel this".

Job statuses: `queued` (waiting), `running`, `done`, `failed`, `cancelled`.

Phase 0: Setup

What you're doing: Installing the tools and creating the project folder.

You need:

- **Node.js 20 or newer** (runs JavaScript tools like React's builder). Check with `node -v`.
- **Python 3.10 or newer**. Check with `python --version`.
- **Git** (saves versions of your code). Check with `git --version`.
- A code editor such as VS Code.

Steps:

```
bash
mkdir mymodly
cd mymodly
git init
```

Create a `.gitignore` file in `mymodly/` so you don't commit junk:

```
.venv/  
node_modules/  
__pycache__/  
api/uploads/  
api/outputs/  
*.pt  
*.safetensors
```

Habit to build: after every phase, run `git add . && git commit -m "phase N done"`. If you break something later, you can go back.

Phase 1: The backend with a fake AI

Goal: Using only your browser, start a job, watch its progress go from 0 to 100, and download a real `.glb` file.

Why fake AI first? Real models need a GPU, are hard to install, and are slow. If you start there and something breaks, you won't know whether the bug is in your code or in the model. A fake generator that waits a few seconds and returns a sphere lets you build and test *everything else* first. Later you swap in the real one without changing anything else.

Step 1.1: Create a virtual environment

A **virtual environment** is a private box of Python packages for this project, so it doesn't mix with other projects.

```
bash  
  
mkdir api  
cd api  
python -m venv .venv  
source .venv/bin/activate      # Windows: .venv\Scripts\activate
```

You'll see `(.venv)` at the start of your terminal line. That means it's active. You need to activate it every time you open a new terminal.

Step 1.2: Install packages

```
bash  
  
pip install fastapi "uvicorn[standard]" python-multipart trimesh numpy  
pip freeze > requirements.txt
```

Package	Why
<code>fastapi</code>	The web framework. Turns Python functions into URLs
<code>uvicorn</code>	The server program that actually runs FastAPI
<code>python-multipart</code>	Lets FastAPI accept file uploads (uploads fail without it)
<code>trimesh</code>	Creates and exports 3D shapes
<code>numpy</code>	Math library that trimesh needs

Step 1.3: Create the file structure

```
bash

mkdir -p app/generators extensions uploads outputs
touch app/__init__.py app/generators/__init__.py
```

(On Windows PowerShell, create these folders and empty files by hand or with your editor.)

Step 1.4: Write the run manager: `app/runs.py`

Concept: a Run is one job, like an order ticket. It stores the job's ID, status, progress, and output file location.

Concept: a thread pool with one worker is like having one chef. Jobs wait in line and get cooked one at a time. Real AI models use the GPU, and two at once would run out of memory, so one at a time is the safe default.

Concept: `threading.Event` is a flag we can raise to say "please stop". The generator checks it regularly.

```
python
```

```

import uuid
import time
import threading
from concurrent.futures import ThreadPoolExecutor
from dataclasses import dataclass, field

@dataclass
class Run:
    id: str
    status: str = "queued"
    progress: int = 0
    output_path: str | None = None
    error: str | None = None
    created: float = field(default_factory=time.time)
    cancel: threading.Event = field(default_factory=threading.Event)

class RunManager:
    def __init__(self):
        self.runs: dict[str, Run] = {} # all jobs, by ID
        self.pool = ThreadPoolExecutor(max_workers=1) # one chef

    def submit(self, fn, *args) -> Run:
        run = Run(id=uuid.uuid4().hex)
        self.runs[run.id] = run
        self.pool.submit(self._execute, run, fn, *args)
        return run

    def _execute(self, run, fn, *args):
        if run.cancel.is_set():
            run.status = "cancelled"
            return
        run.status = "running"
        try:
            run.output_path = fn(run, *args)
            run.status = "done"
            run.progress = 100
        except Exception as e:
            if run.cancel.is_set():
                run.status = "cancelled"
            else:
                run.status = "failed"
                run.error = str(e)

```

Step 1.5: Write the fake generator: `app/generators/mock.py`

```
python

import time
import trimesh

def generate(run, image_path, out_dir):
    for i in range(1, 11):
        if run.cancel.is_set():
            raise RuntimeError("cancelled")
        time.sleep(0.3)          # pretend to think
        run.progress = i * 10    # update progress: 10, 20, ... 100

    out_file = f"{out_dir}/{run.id}.glb"
    trimesh.creation.icosphere(subdivisions=3).export(out_file)
    return out_file
```

Notice the shape of this function: it takes `(run, image_path, out_dir)` and returns the path of the file it made. **Every generator, fake or real, must follow this shape.** That is what makes them swappable. This is called an *interface* (a promise about inputs and outputs).

Step 1.6: Write the routes: `app/main.py`

```
python
```

```
import uuid
from pathlib import Path

from fastapi import FastAPI, UploadFile, File, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.responses import FileResponse

from .runs import RunManager
from .generators import mock

BASE = Path(__file__).resolve().parent.parent
UPLOADS = BASE / "uploads"
OUTPUTS = BASE / "outputs"
UPLOADS.mkdir(exist_ok=True)
OUTPUTS.mkdir(exist_ok=True)

app = FastAPI()

# CORS: lets the React app (on port 5173) talk to this server (port 8000).
# Without it the browser blocks the requests.
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173"],
    allow_methods=["*"],
    allow_headers=["*"],
)

manager = RunManager()

def get_run_or_404(run_id: str):
    run = manager.runs.get(run_id)
    if not run:
        raise HTTPException(404, "run not found")
    return run

@app.get("/health")
def health():
    return {"ok": True}

@app.post("/runs")
async def create_run(image: UploadFile = File(...)):
```

```

suffix = Path(image.filename or "").suffix.lower()
if suffix not in {".png", ".jpg", ".jpeg", ".webp"}:
    raise HTTPException(400, "unsupported image type")

path = UPLOADS / f"{uuid.uuid4().hex}{suffix}"
path.write_bytes(await image.read())

run = manager.submit(mock.generate, str(path), str(OUTPUTS))
return {"id": run.id, "status": run.status}

@app.get("/runs/{run_id}")
def get_run(run_id: str):
    r = get_run_or_404(run_id)
    return {"id": r.id, "status": r.status, "progress": r.progress, "error": r.error}

@app.get("/runs/{run_id}/result")
def get_result(run_id: str):
    r = get_run_or_404(run_id)
    if r.status != "done":
        raise HTTPException(409, "not ready yet")
    return FileResponse(r.output_path, media_type="model/gltf-binary")

@app.delete("/runs/{run_id}")
def cancel_run(run_id: str):
    get_run_or_404(run_id).cancel.set()
    return {"ok": True}

```

Why validate the file type? Never trust what users upload. Rejecting anything that isn't an image is your first line of security.

Step 1.7: Run it and test it

From inside `api/` (with the virtual environment active):

```

bash

uvicorn app.main:app --reload --port 8000

```

Open <http://localhost:8000/docs>. FastAPI builds a test page for every route automatically.

1. Click `POST /runs`, then "Try it out", choose any image, and Execute. Copy the `id` from the response.

2. Try `GET /runs/{id}` a few times and watch `progress` climb.
3. When the status is `done`, try `GET /runs/{id}/result` and download the file.
4. Open the `.glb` in any viewer (for example, Windows "3D Viewer" or the online `glTF-viewer` sites) to check it's a sphere.

Phase 1 checklist

- ☐ `/health` returns `{"ok": true}`
- ☐ An upload returns an ID
- ☐ Polling shows progress rising to 100
- ☐ The downloaded `.glb` opens as a sphere
- ☐ Uploading a `.txt` file gives a 400 error
- ☐ Committed to git

If something goes wrong

Problem	Likely cause
<code>ModuleNotFoundError</code>	Virtual environment not active, or you ran <code>uvicorn</code> from the wrong folder (run it from <code>api/</code>)
Upload gives an error about "form data"	<code>python-multipart</code> isn't installed
<code>attempted relative import</code>	You ran <code>python app/main.py</code> instead of the <code>uvicorn</code> command

Phase 2: The React frontend

Goal: A page with an upload button. Click Generate and see a live progress bar.

Concept: React builds pages out of small reusable pieces called *components*. When data changes (like progress going from 40 to 50), React updates only the parts of the page that need to change.

Concept: state is data that the page remembers and re-draws when it changes, such as "current progress".

Step 2.1: Create the project

Open a **second terminal** (leave the backend running in the first). From the `mymodly/` folder:

```
bash
```

```
npm create vite@latest web -- --template react-ts
cd web
npm install
npm install three @react-three/fiber @react-three/drei zustand
npm install -D tailwindcss@3 postcss autoprefixer
npx tailwindcss init -p
```

- **Vite** is the tool that builds and serves your React app fast.
- **TypeScript** is JavaScript with types, which catches many mistakes before you run the code.
- **Tailwind** lets you style using short class names like `p-4 bg-gray-100`.

In `tailwind.config.js`, set `content` to:

```
js
content: ["./index.html", "./src/**/*.ts,tsx"],
```

At the top of `src/index.css`, put:

```
css
@tailwind base;
@tailwind components;
@tailwind utilities;
```

Run the dev server with `npm run dev`. Open <http://localhost:5173>. You should see the Vite starter page.

Step 2.2: API helper: `src/api/client.ts`

This file is the **only** place that knows the backend's address. If it changes, you fix one file.

```
ts
```

```

const BASE = import.meta.env.VITE_API_URL ?? "http://localhost:8000";

export async function createRun(file: File) {
  const form = new FormData();
  form.append("image", file); // "image" must match the parameter name in FastAPI
  const res = await fetch(`${BASE}/runs`, { method: "POST", body: form });
  if (!res.ok) throw new Error(await res.text());
  return res.json() as Promise<{ id: string; status: string }>;
}

export const getRun = (id: string) =>
  fetch(`${BASE}/runs/${id}`).then((r) => r.json());

export const cancelRun = (id: string) =>
  fetch(`${BASE}/runs/${id}`, { method: "DELETE" });

export const resultUrl = (id: string) => `${BASE}/runs/${id}/result`;

```

Step 2.3: The polling hook: `src/hooks/useRun.ts`

Concept: polling means asking "is it done yet?" on a timer. It's like a kid in the back seat asking "are we there yet?" every second.

Concept: a hook is a reusable piece of React logic. `useRun` hides all the timer code so your page stays clean.

ts

```

import { useEffect, useRef, useState } from "react";
import { createRun, getRun } from "../api/client";

const FINISHED = ["done", "failed", "cancelled"];

export function useRun() {
  const [run, setRun] = useState<any>(null);
  const timer = useRef<number | undefined>(undefined);

  const start = async (file: File) => {
    const { id } = await createRun(file);
    setRun({ id, status: "queued", progress: 0 });

    timer.current = window.setInterval(async () => {
      const latest = await getRun(id);
      setRun(latest);
      if (FINISHED.includes(latest.status)) {
        window.clearInterval(timer.current); // stop asking once finished
      }
    }, 1000);
  };

  // cleanup: stop the timer if the page is closed
  useEffect(() => () => window.clearInterval(timer.current), []);

  return { run, start };
}

```

Step 2.4: The Generate page

Replace `src/App.tsx` with:

tsx

```

import { useState } from "react";
import { useRun } from "../hooks/useRun";

export default function App() {
  const [file, setFile] = useState<File | null>(null);
  const { run, start } = useRun();

  return (
    <div className="max-w-xl mx-auto p-6 space-y-4">
      <h1 className="text-2xl font-bold">Image to 3D</h1>

      <input
        type="file"
        accept="image/*"
        onChange={(e) => setFile(e.target.files?.[0] ?? null)}
      />

      <button
        className="px-4 py-2 rounded bg-blue-600 text-white disabled:opacity-50"
        disabled={!file}
        onClick={() => file && start(file)}
      >
        Generate 3D model
      </button>

      {run && (
        <div>
          <p>Status: {run.status}</p>
          <div className="w-full h-3 bg-gray-200 rounded">
            <div
              className="h-3 bg-blue-600 rounded"
              style={{ width: `${run.progress}%` }}
            />
          </div>
          {run.error && <p className="text-red-600">{run.error}</p>}
        </div>
      )}
    </div>
  );
}

```

Phase 2 checklist

- ☐ Choosing a file enables the button

- ☐ Clicking it makes the status change: queued, running, done
- ☐ The progress bar fills up
- ☐ You can see the requests in your browser's DevTools, Network tab (press F12)
- ☐ Committed to git

If something goes wrong

Problem	Likely cause
Red "CORS" error in the browser console	The origin in <code>allow_origins</code> doesn't match the URL in your address bar exactly (<code>localhost</code> versus <code>127.0.0.1</code> counts as different)
"Failed to fetch"	The backend isn't running
Progress never moves	Check the Network tab: is <code>/runs/{id}</code> being called every second?

Phase 3: The 3D viewer

Goal: When the job finishes, the 3D model appears on the page and you can spin it with your mouse.

Concept: `three.js` is the standard JavaScript library for 3D in the browser. **react-three-fiber** lets you use it with React components. **drei** is a toolbox of ready-made helpers (camera controls, lighting).

Concept: GLB is a compact 3D file format that browsers load easily. Think of it as "the JPEG of 3D".

Step 3.1: The viewer component: `src/components/Viewer.tsx`

tsx

```
import { Canvas } from "@react-three/fiber";
import { OrbitControls, Stage, useGLTF } from "@react-three/drei";

function Model({ url }: { url: string }) {
  const { scene } = useGLTF(url);
  return <primitive object={scene} />;
}

export default function Viewer({ url }: { url: string }) {
  return (
    <div className="w-full h-96 border rounded">
      <Canvas camera={{ position: [0, 0, 3] }}>
        <Stage>
          <Model url={url} />
        </Stage>
        <OrbitControls />
      </Canvas>
    </div>
  );
}
```

- `Canvas` is the 3D window.
- `Stage` adds automatic lighting and centering.
- `OrbitControls` lets you rotate (drag), zoom (scroll), and pan (right-drag).

Step 3.2: Use it in `App.tsx`

Add the imports at the top:

```
tsx

import Viewer from "../components/Viewer";
import { resultUrl } from "../api/client";
```

Add this inside the `{run && (...)}` block, below the error line:

```
tsx
```

```

{run.status === "done" && (
  <>
    <Viewer url={resultUrl(run.id)} />
    <a
      className="underline text-blue-600"
      href={resultUrl(run.id)}
      download={`model-${run.id}.glb`}
    >
      Download .glb
    </a>
  </>
)}

```

Phase 3 checklist

- ☐ After "done", a sphere appears
- ☐ You can rotate and zoom it
- ☐ The download link works
- ☐ Committed to git

Take a moment to celebrate. You now have a complete, working pipeline: upload, background job, progress, 3D result. **This is your minimum viable project.** If everything else went wrong, you'd still have something you can demo. Everything after this makes it better.

Small upgrades to try (optional, 1 to 2 hours)

- Show a preview of the uploaded image.
- Add a Cancel button that calls `cancelRun(run.id)`.
- Keep a history list of past runs in a Zustand store.

Phase 4: The extension system

Goal: Drop a folder into `api/extensions/` and a new generator shows up in the app, without changing the core code.

Concept: a plugin (extension) is a small add-on that plugs into a bigger program. Modly works this way: each AI model is its own separate add-on. That is why it's easy to add new models.

Why bother? Without extensions, adding a model means editing your main code every time. With them, the core stays the same and you only add new folders.

Step 4.1: Define what an extension looks like

Each extension is a folder with two files:

```
api/extensions/mock-cube/  
├─ manifest.json      # the ID card: name, version, what it does  
└─ generator.py       # the code: must define generate(run, image_path, out_dir)
```

manifest.json:

```
json  
  
{  
  "id": "mock-cube",  
  "name": "Mock Cube",  
  "version": "0.1.0",  
  "type": "model",  
  "entry": "generator.py"  
}
```

generator.py:

```
python  
  
import time  
import trimesh  
  
def generate(run, image_path, out_dir):  
    for i in range(1, 6):  
        if run.cancel.is_set():  
            raise RuntimeError("cancelled")  
        time.sleep(0.4)  
        run.progress = i * 20  
        out = f"{out_dir}/{run.id}.glb"  
        trimesh.creation.box(extents=(1, 1, 1)).export(out)  
        return out
```

Also move your Phase 1 sphere into `api/extensions/mock-sphere/` with its own manifest so both appear in the list.

Step 4.2: The loader: `app/extensions.py`

```
python
```

```

import json
import importlib.util
from pathlib import Path

EXT_DIR = Path(__file__).resolve().parent.parent / "extensions"

def load_all() -> dict:
    """Scan the extensions folder and load every valid extension."""
    found = {}
    if not EXT_DIR.exists():
        return found

    for folder in EXT_DIR.iterdir():
        manifest_file = folder / "manifest.json"
        if not manifest_file.exists():
            continue
        try:
            manifest = json.loads(manifest_file.read_text())
            spec = importlib.util.spec_from_file_location(
                manifest["id"], folder / manifest["entry"]
            )
            module = importlib.util.module_from_spec(spec)
            spec.loader.exec_module(module)
            found[manifest["id"]] = {
                "manifest": manifest,
                "generate": module.generate,
            }
        except Exception as e:
            print(f"Skipping {folder.name}: {e}") # one bad plugin shouldn't crash the whole app
    return found

```

Step 4.3: Wire it into `main.py`

```

python

from . import extensions

installed = extensions.load_all()

@app.get("/extensions")
def list_extensions():
    return [e["manifest"] for e in installed.values()]

```

Change `create_run` to accept a choice:

```
python

from fastapi import Form

@app.post("/runs")
async def create_run(image: UploadFile = File(...), extension_id: str = Form(...)):
    ext = installed.get(extension_id)
    if not ext:
        raise HTTPException(400, "unknown extension")
    # ... save image as before ...
    run = manager.submit(ext["generate"], str(path), str(OUTPUTS))
    return {"id": run.id, "status": run.status}
```

Step 4.4: Update the frontend

- In `client.ts`, add `getExtensions()` and send `extension_id` in the form data.
- On the Generate page, load the list once and show a `<select>` dropdown.

Step 4.5 (stretch): install from a URL

Add `POST /extensions/install` that:

1. Downloads a repo as a zip into `extensions/.staging/`.
2. Checks that `manifest.json` exists and is valid.
3. Moves the folder into `extensions/<id>/`.
4. Reloads the extension list.

Security warning (important!)

An extension is real Python code, and loading it runs it. A malicious extension could delete your files. For a college project:

- Only install from sources you trust.
- Check that `manifest["id"]` contains only letters, numbers, and dashes, so nobody can sneak in a path like `../../something`.
- Make sure extracted files stay inside `extensions/`.
- Mention this in your report as a known risk and a future improvement (for example, running extensions in a sandbox).

Phase 4 checklist

- ☐ `GET /extensions` lists both mock extensions
- ☐ The dropdown shows them
- ☐ Choosing "Mock Cube" produces a cube, and "Mock Sphere" produces a sphere

- ☐ A broken extension doesn't crash the server
 - ☐ Committed to git
-

Phase 5: A real AI model (needs a GPU)

Goal: Turn a real photo into a real 3D mesh.

Skip this phase if your computer has no NVIDIA GPU. A working app with a clear extension interface, plus a real model demonstrated on a friend's or lab's GPU machine, is a perfectly good MCA deliverable.

How to do it

1. Pick one open-source image-to-3D model (Hunyuan3D 2 Mini is what Modly's official extension uses).
2. Create a new extension folder: `manifest.json`, `generator.py`, and `setup.py`.
3. `setup.py` installs that model's dependencies. **Keep them separate** so the core app stays light.
4. `generator.py` loads the model and implements the same `generate(run, image_path, out_dir)` function as the fake ones. Nothing else in your app changes. This is the payoff of the interface you defined in Phase 1.

Things that will bite you

- **Model files are huge** (often several GB). Download them once and reuse them.
- **GPU out-of-memory errors** are common. Catch them and set a friendly error message on the run.
- **First run is slow** because the model has to load. Load it once and reuse it, not per job.
- **Dependency conflicts.** Models often need specific versions of PyTorch. Test in a separate virtual environment first.

Phase 5 checklist

- ☐ A real photo produces a real mesh
 - ☐ Errors show a readable message instead of crashing
 - ☐ Extension is in its own folder, and the core code didn't change
-

Phase 6: The workflow canvas

Goal: A drag-and-drop editor where users connect boxes: **Image Input**, **Generate Mesh**, **Output**.

Concept: a node graph is boxes ("nodes") connected by lines ("edges"). Each box does one step, and the lines say what goes where. Modly uses this so users can build custom pipelines.

Do this only after Phases 1 to 4 are solid. It looks impressive, but a shaky base makes it useless.

Steps

1. Install React Flow: `npm install @xyflow/react`.
2. Create three node types: **Image Input**, **Generate Mesh** (with an extension dropdown), **Output**.
3. **Validate before running:** every node must be connected, and there must be exactly one input and one output. Show a warning instead of running if the graph is broken.
4. Start simple: a fixed line of three nodes. Add more node types (like "Smooth Mesh") later.
5. On the backend, add `POST /workflow-runs` that takes the graph as JSON, then runs the nodes in order (each node's output becomes the next node's input).

Phase 6 checklist

- ☐ You can drag, drop, and connect nodes
 - ☐ Bad graphs show a warning
 - ☐ A valid graph runs and shows the result
 - ☐ Committed to git
-

Phase 7: Polish

Goal: Make it reliable and presentable.

Task	Why it matters
Logs page	Shows backend errors so you can debug during a demo
Clean up old runs	Runs sit in memory forever. Modly had to fix exactly this kind of memory leak. Delete runs older than an hour
Save history to SQLite	Runs currently disappear when the server restarts
Tests	<code>pytest</code> for the API, a few tests for the React components
README	How to install and run, with screenshots
Architecture diagram	Reuse the diagram from Part 0. Examiners love this
Error messages	Every failure should say something human-readable
Attribution	Credit Modly in the README and the app footer

Simple test example (`api/tests/test_api.py`)

```
python

from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_health():
    assert client.get("/health").json() == {"ok": True}

def test_unknown_run_is_404():
    assert client.get("/runs/doesnotexist").status_code == 404
```

Run with: `pip install pytest httpx` then `pytest`.

Phase 8: Electron integration

Goal: Double-click one app instead of running two servers.

Concept: Electron is a program that bundles a web browser (Chromium) and Node.js so you can ship a website as a desktop app. VS Code, Slack, and Discord are built this way. Your React code doesn't change, and Electron simply *wraps* it.

What Electron will do for you

1. Start the Python server automatically.
2. Wait until it's ready.
3. Open a window showing your React app.
4. Stop the Python server when the window closes.

Step 8.1: Create the Electron folder

```
bash

cd mymodly
mkdir electron
cd electron
npm init -y
npm install -D electron electron-builder
```

Step 8.2: `electron/main.js`

```
js
```

```

const { app, BrowserWindow } = require("electron");
const { spawn } = require("child_process");
const path = require("path");

let pythonProcess;
const isDev = !app.isPackaged;

function startBackend() {
  pythonProcess = spawn(
    "python",
    [ "-m", "uvicorn", "app.main:app", "--host", "127.0.0.1", "--port", "8000"],
    { cwd: path.join(__dirname, "../api") }
  );
  pythonProcess.stderr.on("data", (d) => console.error(String(d)));
}

// Keep asking /health until the server answers (max ~30 seconds)
async function waitForBackend() {
  for (let i = 0; i < 60; i++) {
    try {
      const res = await fetch("http://127.0.0.1:8000/health");
      if (res.ok) return;
    } catch {}
    await new Promise((r) => setTimeout(r, 500));
  }
  throw new Error("Backend did not start");
}

app.whenReady().then(async () => {
  startBackend();
  await waitForBackend();

  const win = new BrowserWindow({ width: 1280, height: 800 });
  if (isDev) {
    win.loadURL("http://localhost:5173");
  } else {
    win.loadURL("http://127.0.0.1:8000"); // backend serves the built React files
  }
});

app.on("before-quit", () => pythonProcess?.kill());

```

Set `"main": "main.js"` in `electron/package.json` and add a script: `"start": "electron ."`.

Step 8.3: Try it in development

1. In `web/`: `npm run dev` (React on port 5173).
2. In `electron/`: `npm start`.

A window opens showing your app, and the Python server started on its own.

Step 8.4: Serve the built React app from FastAPI (for the packaged version)

Build the frontend once: in `web/`, run `npm run build`. Then add to `main.py` (as the **last** thing in the file, after all your routes):

```
python

from fastapi.staticfiles import StaticFiles
app.mount("/", StaticFiles(directory=BASE.parent / "web" / "dist", html=True), name="w
```

Now one server provides both the API and the web pages, which also avoids CORS problems.

Step 8.5: Package it as an installer

Add to `electron/package.json`:

```
json

"build": {
  "appId": "com.yourname.mymodly",
  "productName": "MyModly",
  "extraResources": [
    { "from": "../api", "to": "api", "filter": ["**/*", "!venv/**"] },
    { "from": "../web/dist", "to": "web-dist" }
  ]
}
```

Then run `npx electron-builder`.

The hard part is Python. A packaged app can't assume the user has Python installed. Two options:

Option	Difficulty	Notes
PyInstaller: turn the backend into a single <code>.exe</code> and launch that instead of <code>python</code>	Easier	Good enough for a college project
Embedded Python (what Modly does): ship a Python runtime inside the installer	Harder	Most flexible, more setup

Phase 8 checklist

- ☐ Running Electron in dev starts the backend automatically
 - ☐ Closing the window stops the backend (no leftover Python process in Task Manager)
 - ☐ The built version loads without needing `npm run dev`
 - ☐ (Stretch) An installer builds and runs on a clean machine
-

Suggested timeline (one person, part-time)

Phase	Time
0 to 3 (working demo)	about 1 to 1.5 weeks
4 extensions	about 3 to 4 days
5 real model	3 to 7 days (hardware dependent)
6 workflow canvas	about 1 week
7 polish and report	about 1 week
8 Electron	about 3 to 5 days

These are rough guesses. Adjust to your deadline, and cut Phases 5, 6, and 8 first if you run short.

Top 10 beginner mistakes

- Running the backend from the wrong folder.** Run `uvicorn` from inside `api/`.
 - Forgetting to activate the virtual environment.** Look for `(.venv)` in your terminal.
 - CORS errors.** The allowed origin must exactly match your frontend address.
 - Forgetting `python-multipart`.** File uploads break without it.
 - Running the AI on the request itself.** Always use the job queue.
 - Not committing to git.** Commit after every phase.
 - Committing big files.** Model weights and `node_modules` don't belong in git.
 - Building the fancy features first.** Finish Phases 1 to 3 before anything else.
 - Copy-pasting without understanding.** Type the code and change things to see what breaks. You'll be asked about it in your viva.
 - Ignoring error messages.** Read them from the top. The first line usually tells you what's wrong.
-

Glossary

Term	Plain meaning
API	A list of URLs one program can call to talk to another
Backend	The behind-the-scenes program that does the real work
Frontend	What the user sees and clicks in the browser
HTTP	The language browsers and servers use to talk
Route / endpoint	One URL in your API, like <code>/runs</code>
JSON	A text format for sending data, like <code>{"id": "abc"}</code>
Polling	Asking "is it done yet?" repeatedly on a timer
Thread	One line of work running alongside others in a program
Queue	A line of jobs waiting their turn
Virtual environment	A private box of Python packages for one project
Component (React)	A reusable piece of the page
State (React)	Data the page remembers and re-draws when it changes
Hook (React)	A reusable piece of React logic, like <code>useRun</code>
Mesh	A 3D shape made of many tiny triangles
GLB	A compact file format for 3D models
Interface	A promise about what a piece of code takes in and gives back
Extension / plugin	An add-on that plugs into the main program
Manifest	A small file describing an extension (its ID card)
CORS	A browser safety rule that blocks a page from calling a different address unless the server allows it
Electron	A tool that wraps a web app into a desktop app
MVP	Minimum viable project: the smallest version that works end to end

Where to go from here

Once Phase 3 works, pick a few improvements that make your project yours: user accounts and saved history, live progress with WebSockets instead of polling, shareable links to a model page, or a niche focus such as 3D-printing checks. A few finished features are worth more than many half-finished ones.

Good luck, and remember: **make it work, then make it better.**